

Otterscale

Open-source team mesh network built on Headscale

Product analysis, architecture and design specification

Draft v0.5 · 8 October 2026

What this document is

A decision-ready analysis of building an open-source, multi-tenant, team-oriented mesh VPN on top of Headscale, with a custom console, the official Tailscale clients, a CLI and SDKs. It covers the verdict on open-source viability, the architecture, the tenancy and access model, the key flows, deployment, security and a phased roadmap.

Facts about Headscale and hosting platforms were checked against current sources on 7 Oct 2026 (listed in Appendix C). Anything I could not verify is marked **[verify]** and collected in section 20.

New in v0.5

Official Tailscale clients only. There is no custom client app. People install the unmodified Tailscale apps and point them at their tenant; the Otterscale console gains a **Connect** area (install pages, per-tenant login links, access status). Only one tenant is active at a time, as in the official app's account switcher. Section 15 is rewritten, and the roadmap, summary, diagrams and open questions are updated. Roles, single console and all-in-one mode are unchanged.

Contents

1. Executive summary and verdict	4
What to build first, and what to postpone	4
Scorecard for open-source viability	5
2. Your questions, answered	6
3. Verified facts and design constraints	7
4. Product concept	7
Personas	7
Mental model: networking like a cloud account	7
In scope and out of scope	8
5. System architecture	8
Key design decisions	8
Technology choices	9
6. Tenants, environments and groups	11
Two ways to do multi-tenancy	11
How environments work (prod, uat, dev)	11
Groups and what people can see	12
7. Roles, single console and tenant lifecycle	12
Role model	12
First run: becoming the Owner	13
One console, adapted to the role	14
Tenants as virtual spaces: two usage profiles	15
What the Platform area does	15
Tenant lifecycle states	16
Safeguards for powerful roles	16
8. Policy compiler	17
Example output	17
Rules of the compiler	17
9. RBAC and audit log	18
Permission matrix	18
What is audited	18
Integrity and export	19
10. Access requests and time-limited access	19
Design rules	20
11. Guest and share links	21
Controls	21
12. Device onboarding, CLI and agent	22

Recommended method per device type	23
The otter command line	23
13. SDKs, Terraform and Kubernetes (the VPC-like layer)	24
Terraform example	24
Kubernetes and Argo CD	24
14. Analytics	26
15. Clients: the official Tailscale apps	26
How people connect	26
Pointing each official app at your server	27
What you give up, and the workaround	27
Risks to plan for	27
When to revisit building your own client	27
16. Deployment and packaging	28
Where each part can run	28
Single-container mode (all-in-one)	30
Quickstart Compose sketch	31
Operations	31
17. Security and threat model	32
18. Roadmap	32
19. Is this good for open source? Deep analysis	33
Landscape	33
Why it can work	34
What can go wrong	34
Licensing and governance	34
Sustainability options	35
20. Open questions and verification checklist	35
Appendix A: control API sketch	36
Appendix B: repository layout	38
Appendix C: sources checked	38

1. Executive summary and verdict

The idea: keep Headscale as the coordination engine (so any official Tailscale-compatible client works), and build everything teams actually miss around it: a polished console, tenants and environments (prod, uat, dev), groups that control who can see which devices, RBAC and an exportable audit log, time-limited access requests, guest share links, analytics, a CLI/agent for servers, SDKs and a Kubernetes operator, and a Connect area that onboards people onto the official Tailscale apps (no custom client). One **role-based console** serves everyone: a single Owner (the first login) adds super admins, super admins create isolated tenant spaces (for example prod, uat, dev) and assign their admins, and an **all-in-one image** can run many tenants inside a single container.

Verdict: yes, this is a good open-source product, conditionally

Headscale is a control server only. The maintainers do not ship a web UI, and a single instance serves one tailnet. Community UIs exist (Headplane is the most complete), but they are mainly administration consoles. A package that adds **multi-tenancy, environments, approval-based temporary access, guest links and audit** for small teams is a real gap. The condition is discipline about scope: ship the control platform and CLI first, and use the official Tailscale clients instead of building your own apps.

What to build first, and what to postpone

Piece	Value to users	Cost / risk	Recommendation
Console + policy compiler + groups + environments	Very high	Medium	Phase 1, the core of the product
RBAC, audit log, onboarding flows	Very high	Medium	Phase 1
Multi-tenancy (instance per tenant) + role-based console with Owner, super admins, tenant admins	High (SaaS, MSPs, agencies)	Medium-high (orchestration, platform security)	Phase 2
Access requests, time-limited grants, guest links	Very high, your clearest differentiator	Medium	Phase 3
otter CLI/agent, Terraform, SDKs	High for DevOps teams	Low-medium	Phase 4
Kubernetes operator (Argo CD, services)	High for platform teams	Medium	Phase 5
Connect area: install pages, per-tenant login links, MDM profiles	High: smooth onboarding on the official apps	Low	Phase 6
Custom client apps (desktop, Android, iOS)	Low: official apps already work	Very high (updates, signing, Apple rules)	Out of scope; revisit only with strong demand

Scorecard for open-source viability

Dimension	Rating	Why
Real problem / demand	Strong	Teams want self-hosted, no-per-seat mesh networking with a usable UI and access control.
Differentiation	Medium-strong	Headplane already covers admin CRUD. Multi-tenancy, approvals, guests, audit and analytics are the gap.
Technical feasibility	Strong	Headscale now has policy tests, grants and an API that partly mirrors Tailscale's. All differentiators sit in your layer.
Maintenance burden	Medium-high	You track Headscale releases (strict upgrade path) and Tailscale client changes.
Dependency risk	Medium	Protocol compatibility is maintained by Headscale, not you. Pin versions and run compatibility CI.
Community potential	Strong	Homelab and small-startup audiences are large, vocal and used to contributing to Headscale ecosystem tools.
Sustainability	Medium	Needs a model: managed hosting, support, or open-core enterprise features (section 19).

2. Your questions, answered

Your question	Short answer	Section
Add phones, laptops, servers and other devices?	Yes. Headscale supports Linux, BSD, Windows, macOS, iOS, tvOS and Android clients. Servers and RPi use the CLI/agent; containers and pods use sidecars or the operator.	12, 13
Groups where members only see their group's devices?	Yes, through policy. Rules are compiled from your group and environment model; devices a user cannot reach should not appear in their device list. Confirm this netmap behaviour in integration tests [verify].	6
Tenants / prod, uat, dev?	Yes, as two levels: a tenant (organisation) is a Headscale instance; an environment is a tag plus rules inside it. For strict isolation, an environment can also be its own instance.	6
Add services like Argo CD or Kubernetes?	Yes. Tagged proxy/subnet-router nodes inside the cluster, managed by your own operator.	13
Multi-tenancy?	Yes, by orchestrating one Headscale instance per tenant, because one Headscale instance serves exactly one tailnet.	6, 16
A single role-based UI with one Owner who adds super admins, who create tenants (prod, uat, dev spaces) and their admins?	Yes. One console adapts to the role. The Owner is the first login and the only one who adds or removes super admins; super admins create and manage tenant spaces and assign each space's admins. Permissions are enforced by the API, not just hidden in the UI.	7, 9
One Docker container running N tenants?	Yes, as N separate Headscale processes under a supervisor inside one container (not one process serving many tenants). Best for small installs; the same model scales out to containers or pods later.	16
RBAC, audit log, access requests, guest links?	Headscale has none of these as products, so they live in your control platform and compile down to policy.	9, 10, 11
Analytics?	Yes for inventory, activity, adoption, access workflow and health. Connection-level flow logs are not available from Headscale; an optional agent can sample them.	14
Our own client app, multiple accounts, tenants listed per account?	Decision: use the official Tailscale apps only. They support several accounts with one active at a time. The Otterscale console lists each person's tenants and gives the login link for each.	15
Do I need a Docker image? Run it on Railway, AWS, Render?	Yes, ship images. Railway and similar PaaS can host the HTTP control platform, but not DERP/STUN (needs inbound UDP). With the all-in-one image, tenant processes run inside the container, so a PaaS with a persistent volume can host many tenants [verify volume support].	16
A CLI like Tailscale's for servers and RPi?	Yes, otter plus an optional background agent.	12
An SDK for prod, k8s, containers, like AWS VPC?	Yes: Terraform provider, Go/Python/TypeScript SDKs, a Go embeddable library and a Kubernetes operator with CRDs.	13

3. Verified facts and design constraints

These findings shape the design. Sources are in Appendix C.

Fact (checked 7 Oct 2026)	Implication for Otterscale
Headscale's latest release is v0.29.3 (29 Jul 2026), BSD-3-Clause.	Permissive licence: you can build and redistribute on top of it. Pin to a tested version.
One Headscale instance serves one tailnet; multi-tenant setups need multiple instances.	Tenants map to instances; build an orchestrator (sections 6, 7, 16).
No official web UI. Community UIs include headscale-ui, Headplane, headscale-admin, ouroboros (self-service) and headscale-console (WebAssembly SSH/VNC/RDP).	Study them; Headplane is the benchmark. ouroboros and headscale-console show demand for self-service and browser access.
v0.29 supports Tailscale grants alongside ACLs, a tests block for policy tests (beta) and SSH check actions.	Your policy compiler can validate before applying and can use modern policy syntax.
The v2 HTTP API at /api/v2 is partly compatible with the Tailscale API (Terraform/OpenTofu provider, tscli, official Go client).	Integrate through the API, not the CLI. You may reuse existing tooling for part of your SDK story [verify coverage of policy endpoints].
Headscale aims to support the last 10 Tailscale client releases; v0.29 requires client v1.80.0 or newer.	Users must keep official clients inside that window; the console shows each device's client version and warns when it is too old. Compatibility CI against several client versions is mandatory.
The maintainers do not support or encourage running Headscale in containers or behind reverse proxies; proxies must handle WebSockets for the client protocol.	Heavy integration testing of your own images and proxy config is on you. Document a reference setup.
Headscale's FAQ recommends SQLite and describes a central scaling bottleneck when certain scenarios overlap.	Small per-tenant instances with SQLite fit the design, and tenant sharding limits blast radius.
v0.29 enforces a strict upgrade path (no skipping minor versions).	The orchestrator must upgrade tenants stepwise, canary first.
Tailscale's Kubernetes operator relies on OAuth against Tailscale's control plane; a Headscale feature request remains open, though one packaging charm advertises Headscale support.	Do not depend on it. Build your own operator against your API [verify before deciding].
Railway routes no inbound UDP; it offers TCP proxy only.	DERP/STUN relays must run on plain VMs. Render and other PaaS: check before relying on them [verify].
Tailscale's own control plane is proprietary and not self-hostable.	This is your market; position as the open, self-hostable team layer.

4. Product concept

Personas

Persona	Needs
Founder / CTO of a small startup	Zero per-seat cost, simple setup in an afternoon, confidence that prod is locked down.
DevOps / platform engineer	Environments, groups, Terraform, Kubernetes integration, audit trail, no snowflake config.
Developer	Install, sign in with company SSO, see only what they should, request access when needed.
Contractor / guest	Temporary access to exactly one machine, with nothing else visible.
Security / compliance reviewer	Who had access to what, when, who approved it; exportable evidence.
MSP / agency (multi-tenant operator)	Many customer networks from one console with strict separation.

Mental model: networking like a cloud account

Your users already understand AWS. Reusing that vocabulary lowers the learning curve and gives SDK/Terraform design a familiar shape.

Cloud concept	Otterscale concept	Implemented as
Account / organisation	Tenant	One Headscale instance + control-plane records
VPC	Environment (prod, uat, dev)	A tag (e.g. tag:prod) plus rules; optional dedicated instance for strict isolation
IAM group	Group	A group in the policy, mapped from IdP groups
Security group rule	Access rule	ACL / grant lines compiled from intent
Resource tags	Tags	Headscale tags set via tag owners and scoped auth keys
Temporary credentials (STS)	Access grant	Time-boxed rule with expires_at, removed by the reaper
Pre-signed URL	Share link	Single-use token creating a guest user, ephemeral key and one narrow rule

In scope and out of scope

- **In scope:** a control platform, console, policy engine, workflows, audit, analytics, CLI/agent, SDKs, operator, a Connect onboarding area for the official clients, deployment packaging.
- **Out of scope:** reimplementing the Tailscale protocol, writing a new WireGuard data plane, or forking Headscale. You depend on Headscale and the open client core, and contribute upstream where gaps appear. Building custom client apps is also out of scope.

5. System architecture

Otterscale is a set of stateless services in front of per-tenant Headscale instances. All differentiating logic sits in your layer, so Headscale can be upgraded or replaced behind a narrow adapter. The diagram on the next page shows every component and its main interaction.

Key design decisions

#	Decision	Reason
1	Integrate through Headscale's API (gRPC/REST v2), never by shelling out to the CLI.	Typed, versionable, testable; works across hosts.
2	Intent-based policy: store groups, environments and rules in your DB, compile to HuJSON.	Humans and the UI edit intent; compilation is deterministic, testable and reversible.
3	One Headscale instance per tenant.	Hard isolation, separate IP ranges and policy, fits Headscale's single-tailnet design.
4	Own database for what Headscale lacks: roles, audit, requests, grants, notes, analytics.	Headscale stores network state, not product state.
5	Stay compatible with official Tailscale-protocol clients.	No custom client apps; adoption never blocks on app-store work.
6	Detect changes by polling plus event hooks, then push to the browser via SSE/WebSocket.	Headscale offers no first-class event stream [verify]; hide that behind an internal event bus.
7	Stateless services, Postgres for control data, SQLite per tenant instance with backups.	Easy to scale the platform; matches Headscale guidance.
8	One role-based console (single UI module) for every role; platform and tenant permissions are separate scopes enforced by the API.	One product to build and learn; hiding screens is cosmetic, so security rests on server-side authorization.
9	Pluggable orchestrator drivers: process (all-in-one), docker, kubernetes.	Same tenant model from a single container up to a cluster.

Technology choices

Layer	Choice	Notes
Backend services	Go	Same ecosystem as Headscale and Tailscale; single static binaries, easy to ship to RPi.
Control DB	PostgreSQL	Row-level tenant scoping, JSONB for policy snapshots.
Console	TypeScript, React or SvelteKit, Tailwind, shadcn-style components	Graph view with React Flow or Cytoscape; Monaco for the policy editor.
Jobs / scheduler	Postgres-backed queue (e.g. River) or NATS	Avoids another moving part for small installs.
Metrics	Prometheus, optional ClickHouse for long retention	Headscale exposes metrics; community exporters exist.
Auth	OIDC for people; scoped API keys and OAuth-style client credentials for machines	SCIM later for enterprise.
Client core	Open-source tailscaled core (BSD-3)	Reuse the proven network engine instead of writing one.

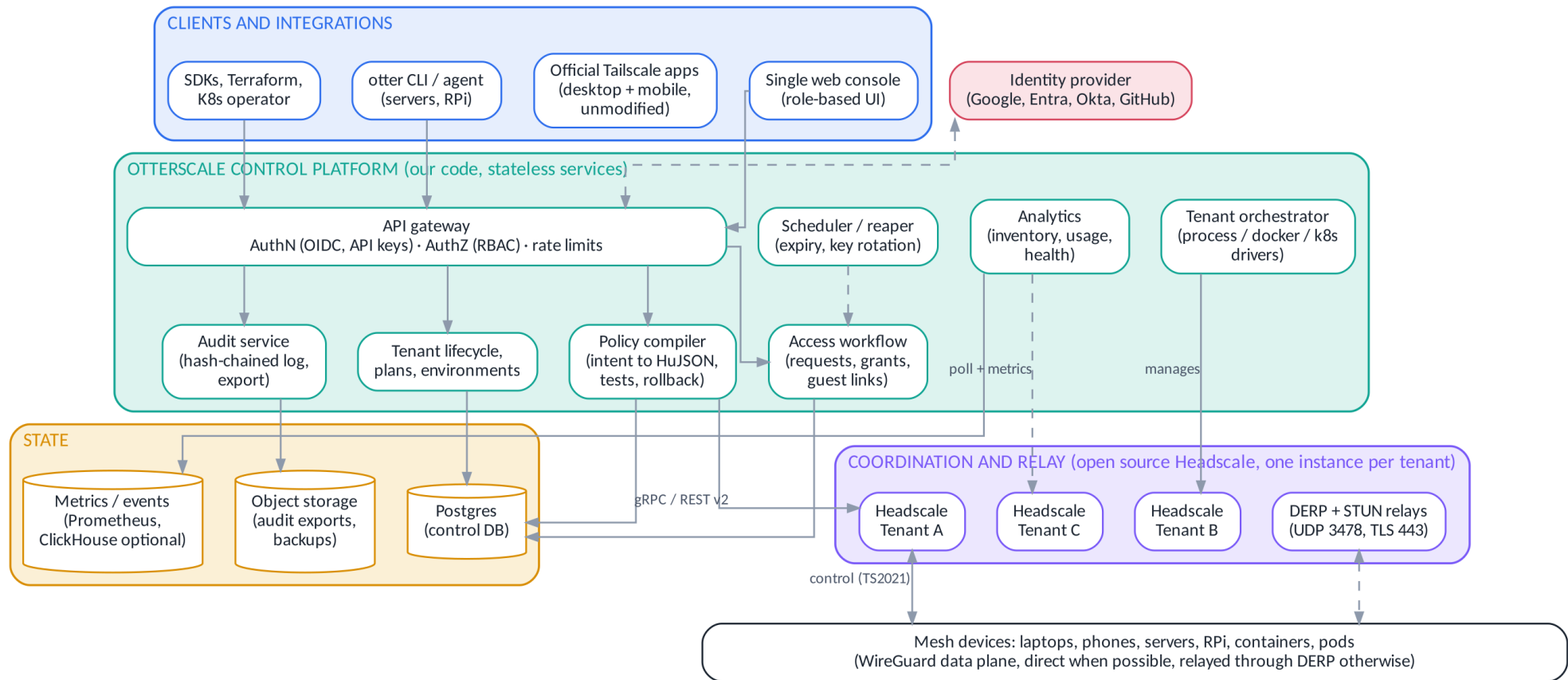


Figure 1. System architecture. Teal is your code, violet is Headscale and relays; arrows show the main call paths.

6. Tenants, environments and groups

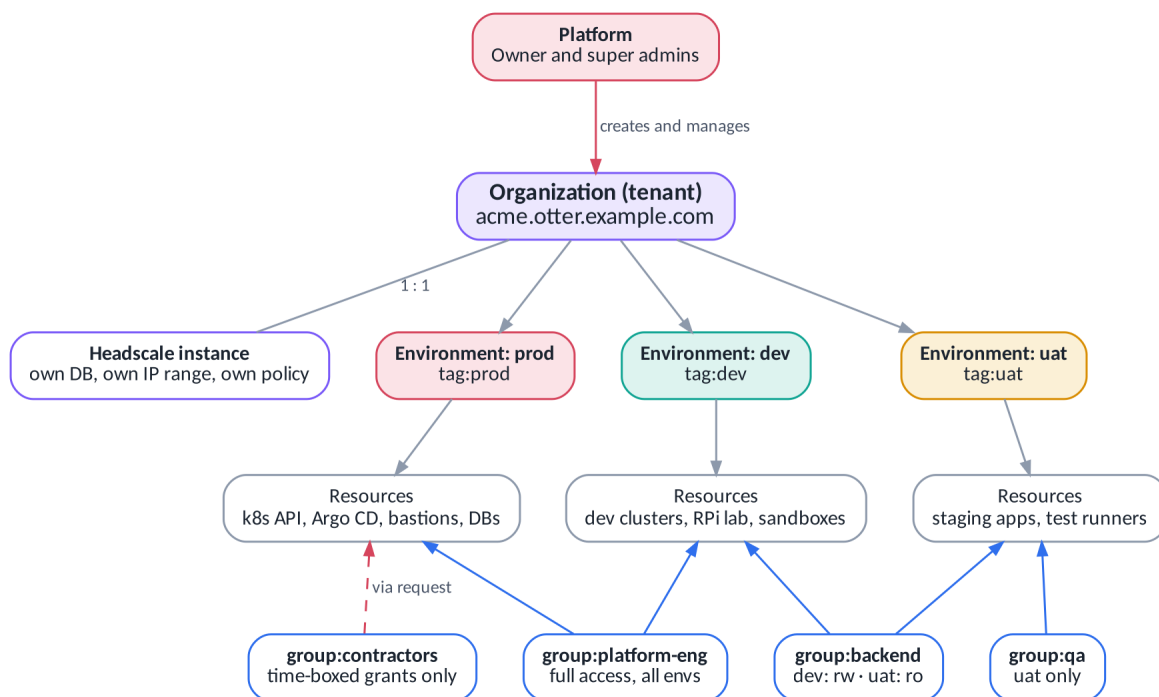


Figure 2. Tenancy model. A tenant maps to one Headscale instance; environments are tags and rules inside it; groups get access per environment.

The platform sits above all tenants: a super admin creates and manages them from the Platform area of the console (section 7).

A tenant is an isolated virtual space. You can use tenants as **your own environments** (tenants named prod, uat and dev, each with its own admins) or as **customers** of a hosted service, with lightweight environments inside each; section 7 compares these profiles.

Two ways to do multi-tenancy

	A. Shared instance (soft)	B. Instance per tenant (hard)
Isolation	Policy only; one mistake can leak across tenants	Separate DB, keys, IP range and policy
Cost	Lowest	A small process per tenant (Headscale is lightweight)
Fit with Headscale	Works against its single-tenant design	Matches it, as multiple instances are the supported pattern
Blast radius	Whole platform	One tenant
Operational complexity	Low	Higher: orchestration, upgrades, backups per tenant
Recommendation	Use only for environments inside one organisation	Use for tenants (as processes, containers or pods; section 16)

How environments work (prod, uat, dev)

- **Default (soft environments):** each device carries one environment tag. Rules decide who may reach which tag, on which ports. Simple and covers most teams.
- **Strict environments (optional):** prod gets its own Headscale instance, so there is no shared policy or IP space with dev. People who need both use two accounts (profiles) in the official Tailscale app and switch between them.
- Tag assignment is controlled: only owners listed in the policy can apply a tag. Auth keys are scoped, e.g. a prod key can only create nodes tagged tag:prod.

Groups and what people can see

A user belongs to one or more groups (mapped from IdP groups or assigned manually). Rules grant a group access to an environment or to specific resources. Peers appear in a device's network map only when policy allows traffic between them, so users should see only the devices their group may reach. Your console additionally filters its own device list by the same rules, so what the UI shows always matches what the network enforces.

IP planning across tenants

Give each tenant instance a **non-overlapping** address prefix from the CGNAT range (for example a /20 per tenant). The official client keeps one account active at a time, so overlap would not break a connection, but distinct prefixes avoid confusion in logs and support, work for people running subnet routers, and keep future options open.

7. Roles, single console and tenant lifecycle

The product ships as **one web console**, a single UI module. What each person sees and can do depends on their role. There is exactly one **Owner** (the first login that set up the platform), the Owner adds **super admins**, super admins create **tenants** (isolated virtual spaces such as prod, uat and dev) and make an **admin for each**, and tenant admins run their own space.

Role model

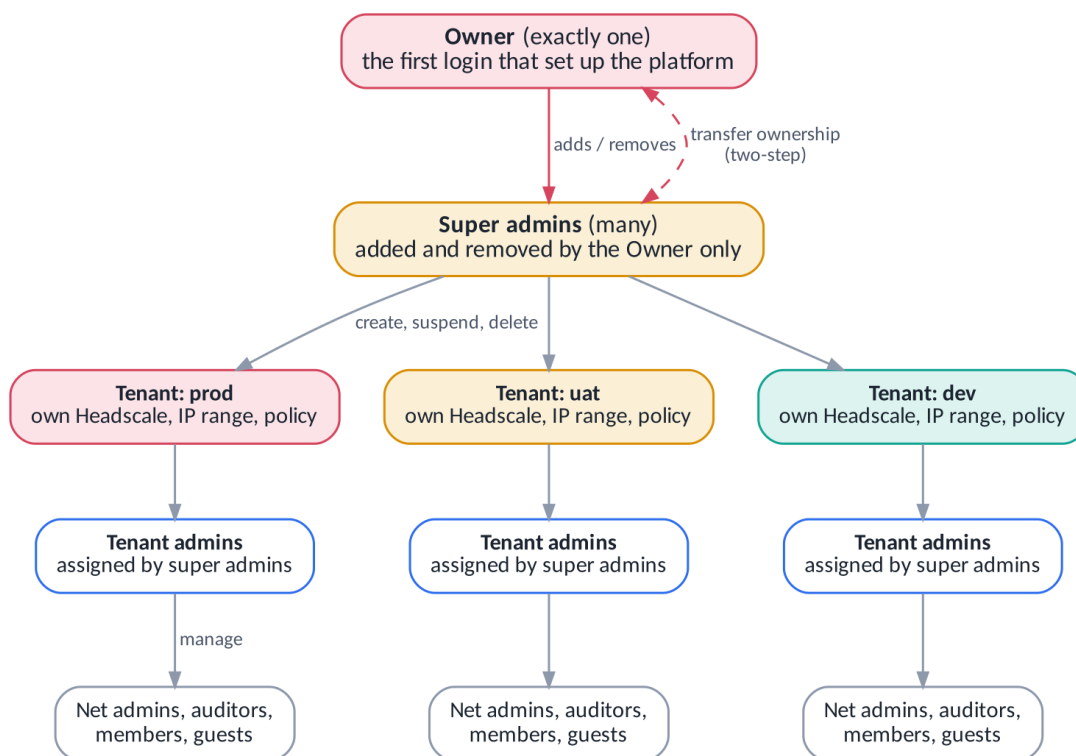


Figure 3. Role hierarchy: one Owner, super admins added by the Owner, tenant spaces created by super admins, and tenant admins assigned per space.

Role	Scope	How many	Created by	Can do
Owner	Platform	Exactly one	First-run setup (first login)	Everything a super admin can; add and remove super admins; transfer ownership; platform settings (domain, SSO, relays, backups).
Super admin	Platform	Many	The Owner only	Create, suspend, resume and delete tenants; assign and remove tenant admins; plans and quotas; upgrades; fleet health; platform audit. Cannot add or remove super admins or the Owner.

Role	Scope	How many	Created by	Can do
Tenant admin	One tenant (assigned per tenant)	One or more per tenant	Super admins	Run that tenant: users, groups and roles below admin level, environments, rules, approvals, devices, audit.
Net admin	Tenant (optionally limited to environments)	Many	Tenant admin	Edit rules, devices, auth keys and approvals for assigned environments.
Auditor	Tenant	Many	Tenant admin	Read-only access to audit log and exports.
Member	Tenant	Many	Tenant admin or IdP groups	Own devices, access requests, share links for own devices.
Guest	One device	Via share link	Members and admins	Reach one device for a limited time.

Rules that keep this safe

- **Exactly one Owner at all times**, enforced by a database constraint. Transferring ownership is a deliberate two-step action: the Owner nominates a super admin, who accepts with MFA; the former Owner becomes a super admin.
- **Only the Owner adds or removes super admins**. Super admins cannot promote themselves or each other, and cannot touch the Owner. Warn when only one super admin exists.
- **Roles are scoped**. A person can be tenant admin in dev and a plain member in prod. Platform roles carry no tenant data rights by themselves: to work inside a tenant a super admin is assigned as a tenant admin there (visible in that tenant's audit log). An install setting can grant super admins implicit tenant-admin rights, sensible for a single organisation and off by default for hosted use.
- **Recovery**: the Owner receives recovery codes at setup; if locked out, a host-level reset command (requires shell access to the server) can re-enrol the Owner.
- **Every role change is audited**; changes to tenant roles are also mirrored into that tenant's audit log.

First run: becoming the Owner

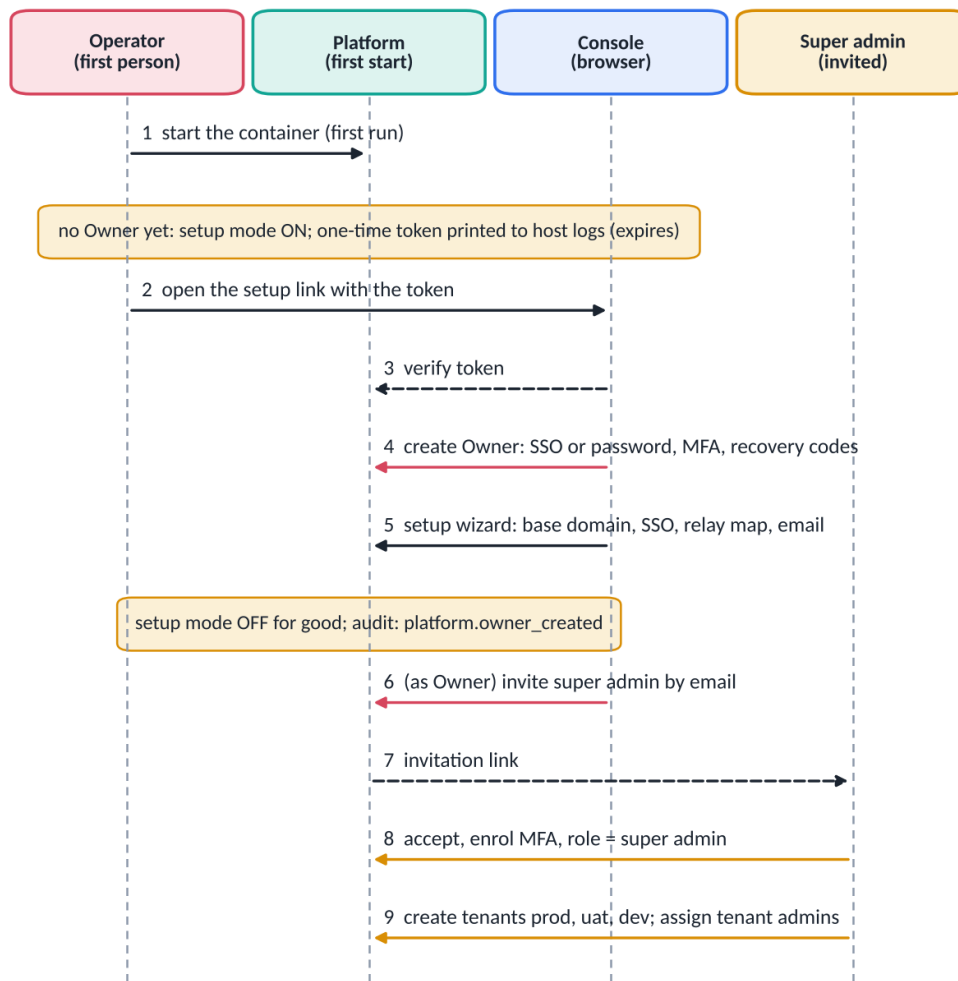


Figure 4. First-run setup: the first login becomes the Owner, then invites super admins, who create tenants and assign their admins.

- **Setup mode exists only while no Owner exists.** A one-time token is printed to the host logs (or set through an environment variable), expires after a short time, and can only be regenerated from the host shell.
- **No default password.** The Owner signs in with SSO or a password plus mandatory MFA and stores recovery codes before the wizard continues.
- **The wizard sets platform basics:** base domain, SSO provider, relay map, outgoing email, default Headscale version. Once finished, setup mode is switched off permanently and recorded in the audit log.

Protect the setup link

Until the Owner is created, whoever holds the setup link could claim the platform. Keep it out of public logs and chat, bind it to a short lifetime, and show a clear "setup mode is on" warning on the health endpoint for monitoring.

One console, adapted to the role

After login the API returns the person's role grants: platform roles and, per tenant, tenant roles. The console builds its navigation and routes from those capabilities, and a tenant switcher in the top bar lists every space the person belongs to (for example prod, uat, dev) with a role badge. Entering the Platform area asks for step-up MFA and shows a clear banner. **Hiding a screen is only cosmetic:** the API authorizes every request on the server, using roles read from the database rather than from the login token.

Console area	Owner	Super admin	Tenant admin	Net admin	Auditor	Member	Guest
Platform: tenants, capacity, relays, platform audit	Yes	Yes	-	-	-	-	-

Console area	Owner	Super admin	Tenant admin	Net admin	Auditor	Member	Guest
Platform: super admins, ownership, settings	Yes	-	-	-	-	-	-
Workspace: users, roles, groups, policy, approvals	-*	-*	Yes	Partly	-	-	-
Network: devices, auth keys, share links	-*	-*	Yes	Yes	Read	Own	-
Audit and analytics (tenant)	-*	-*	Yes	Partly	Yes	-	-
My access: my devices, add device, request, share	-*	-*	Yes	Yes	Yes	Yes	One device

* Only if the person also holds a role in that tenant, or the install grants super admins implicit tenant-admin rights.

Tenants as virtual spaces: two usage profiles

	A. One organisation	B. Many customers (hosted service or MSP)
What a tenant is	A virtual space such as prod, uat or dev	One customer organisation
Inside a tenant	Groups and rules; environments usually unnecessary	Optional lightweight environments (tags and rules, section 6)
Isolation	Strongest separation between prod and dev: separate policy and IP space	Separate instance per customer
Who administers	Tenant admins assigned per space by super admins	Customer's own tenant admins
Super admin rights in tenants	Implicit tenant-admin rights usually on	Off by default; consented support access
People in several spaces	Role per space; switch space in the console, and switch account in the Tailscale app	Usually one tenant per person

People who work in several spaces hold a role in each. In the console they switch space from the top bar. On a device they use the official Tailscale app's account switcher, which keeps one space connected at a time; the console's Connect area shows the right login link for each space.

What the Platform area does

Area	Capabilities
Admins (Owner only)	Invite, remove and review super admins; transfer ownership; recovery settings.
Tenant lifecycle	Create tenant (name and slug, plan and quotas, first tenant admins, relay region), suspend, resume, delete with grace period and data export.
Fleet health	Per-tenant status (provisioning, active, suspended), Headscale up or down, device counts, last policy apply, error rate, resource use.
Capacity	Tenants per container or host, CPU and memory per tenant process, free port range, IP prefix allocation; warnings before limits are reached.
Upgrades	Headscale version per tenant; canary tenants first; stepwise upgrades that respect Headscale's strict upgrade path; rollback to the last snapshot.
Relays	Manage the DERP/STUN fleet, regions and health; publish the relay map to tenants.
Plans and quotas	Limits on devices, users, environments, retention; optional billing hooks and usage export.
Support access	Time-boxed, view-only access to a tenant, with a tenant admin's consent or an alerted break-glass path.
Platform audit	Every platform action, searchable and exportable; mirrored into the affected tenant's own audit log.
Settings (Owner only)	Base domain, SSO provider, email, backup targets, default Headscale version, super-admin rights policy.

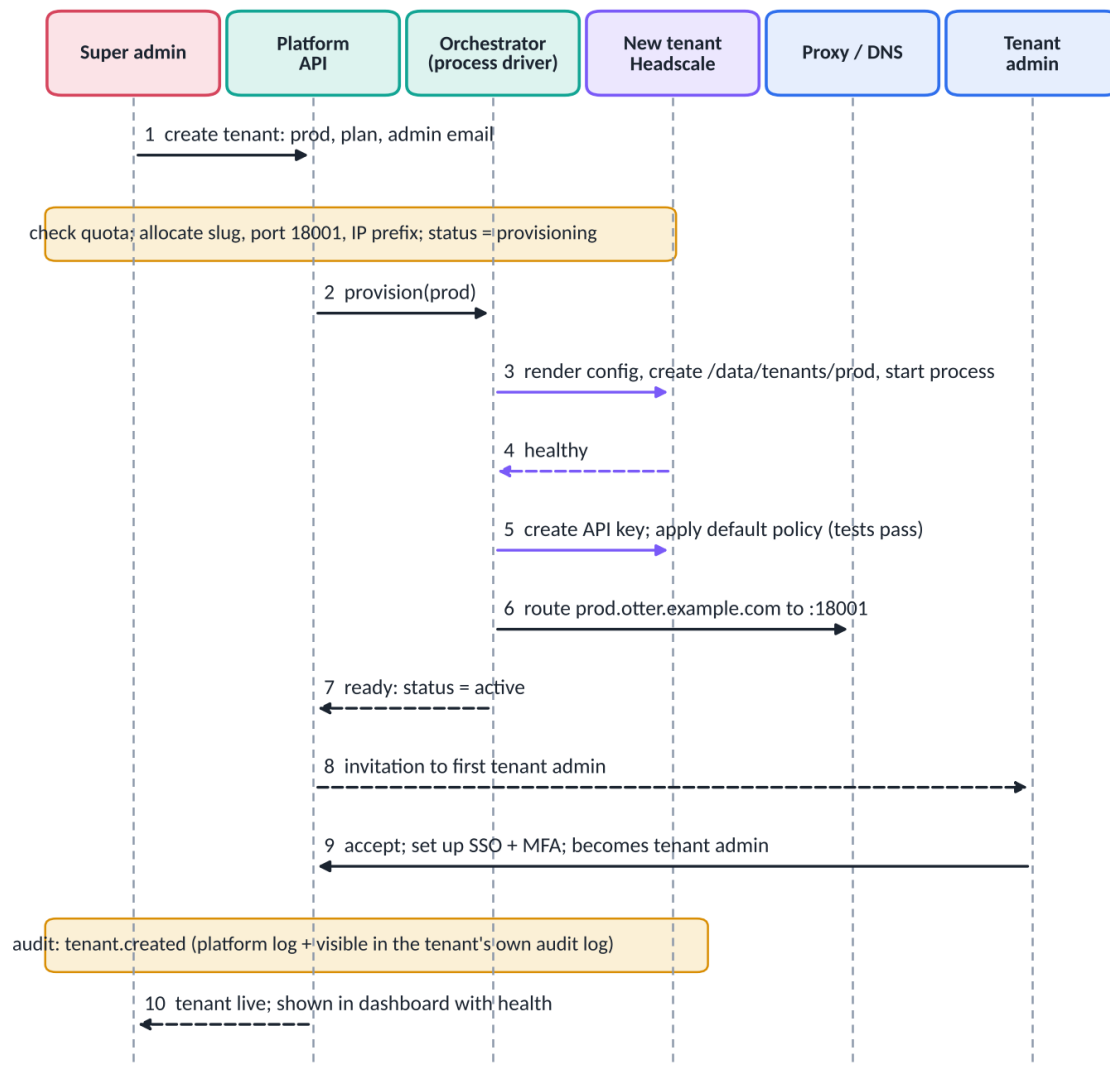


Figure 5. Creating a tenant from the Platform area: allocate, provision, route, invite its admin.

Tenant lifecycle states

State	Meaning	What happens
provisioning	Being created	Resources allocated, process or container started, default policy applied. Failures roll back and release the port and prefix.
active	Normal operation	Console and device logins work; policy changes apply.
suspended	Paused by operator or billing	New logins and policy changes blocked; data kept. Whether existing device connections continue should be an explicit setting [verify behaviour].
pending_delete	Grace period (for example 14 days)	A tenant admin can export data or cancel; backups retained.
deleted	Removed	Process stopped, volume data erased after retention, DNS and port released, audit stub kept.

Safeguards for powerful roles

Super admins can create tenants and, because the platform writes each tenant's policy, could in principle grant access inside one. That power must be bounded and visible.

- **Strong sign-in:** mandatory MFA, hardware keys recommended for the Owner and super admins, short sessions, step-up MFA for the Platform area, optional IP allow-list or a separate hostname for it.

- **No standing access to tenant contents** in hosted setups: support sessions are view-only, time-boxed and need a tenant admin's consent; break-glass is allowed but raises an alert and a prominent audit entry.
- **Transparency:** every platform action that touches a tenant appears in that tenant's own audit log, so tenant admins can see what the operator did.
- **Two-person rule** for deleting tenants, bulk upgrades and changing super admins.
- **Least privilege:** the per-tenant Headscale API keys are held in a KMS and never shown in the UI.
- **Honest limit:** traffic between devices is end-to-end encrypted with keys held on the devices, so the operator cannot read it, but a malicious operator could still change policy. For hosted offerings, publish this trust model; for self-hosters the operator and tenant admins are usually the same team.

8. Policy compiler

The compiler turns your intent records into the policy file Headscale evaluates. It is the most important component: every access feature uses it.

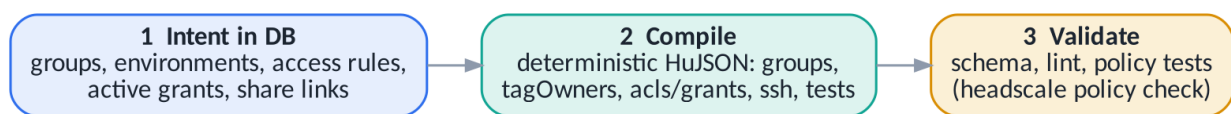


Figure 6a. Policy pipeline, steps 1 to 3.

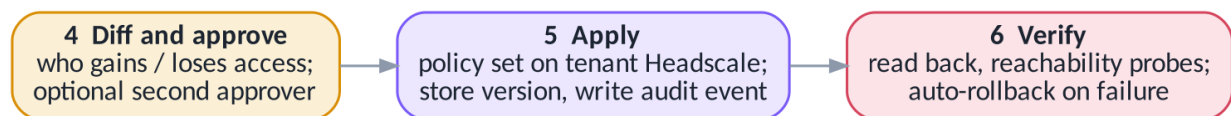


Figure 6b. Policy pipeline, steps 4 to 6.

Example output

```
// generated by otterscale, version 412, sha256 9f3c... (do not edit by hand)
{
  "groups": {
    "group:platform-eng": ["alice@acme.com"],
    "group:backend":      ["bob@acme.com", "carol@acme.com"],
    "group:qa":           ["qa1@acme.com"]
  },
  "tagOwners": {
    "tag:prod": ["group:platform-eng"],
    "tag:uat":  ["group:platform-eng"],
    "tag:dev":  ["group:platform-eng", "group:backend"]
  },
  "acls": [
    {"action": "accept", "src": ["group:platform-eng"], "dst": ["tag:prod:*", "tag:uat:*", "tag:dev:*"]},
    {"action": "accept", "src": ["group:backend"],      "dst": ["tag:dev:*", "tag:uat:80,443"]},
    {"action": "accept", "src": ["group:qa"],           "dst": ["tag:uat:*"]},
    // active time-boxed grant, expires 2026-10-08T13:00:00Z (request #318, approved by alice)
    {"action": "accept", "src": ["bob@acme.com"],      "dst": ["tag:argocd-prod:443"]}
  ],
  "tests": [
    {"src": "qa1@acme.com", "accept": ["tag:uat:443"], "deny": ["tag:prod:22"]}
  ]
}
```

Illustrative only. Validate exact syntax against the Headscale version you pin, using its policy check command, and prefer grants where your pinned version supports what you need.

Rules of the compiler

- **Deterministic output:** the same intent always yields byte-identical HuJSON, so diffs are meaningful.

- **Generated tests block:** every group gets positive and negative assertions (for example, qa can reach uat but not prod). A failing test blocks the apply.
- **Versioned and reversible:** every applied policy is stored with author, time and hash; rollback re-applies a previous version and is itself audited.
- **Diff preview:** before applying, show who gains and who loses access in plain words.
- **Debounce:** batch changes within a few seconds so bursts of grant expiries or approvals do not cause policy-reload storms.
- **Lint:** warn about wildcard destinations, rules with no matching devices, groups with no members and tags nobody owns.

9. RBAC and audit log

Roles and the single console are introduced in section 7. This section gives the full permission matrix and the audit rules. Platform roles (Owner, super admin) and tenant roles are separate scopes, and every check is made by the API on the server.

Permission matrix

Permission	Owner	Super admin	Tenant admin	Net admin	Auditor	Member	Guest
Add or remove super admins, transfer ownership, platform settings	Yes	-	-	-	-	-	-
Create, suspend, delete tenants; plans and quotas	Yes	Yes	-	-	-	-	-
Assign or remove tenant admins	Yes	Yes	-	-	-	-	-
Fleet health, capacity, relays, upgrades, platform audit	Yes	Yes	-	-	-	-	-
Manage users, groups and roles below admin level in a tenant	-	Opt.	Yes	-	-	-	-
Edit environments, access rules, policy	-	Opt.	Yes	Assigned envs	-	-	-
Approve access requests	-	Opt.	Yes	Per env	-	-	-
Create share links	-	Opt.	Yes	Yes	-	Own devices	-
View tenant audit log and exports	-	Opt.	Yes	-	Yes	-	-
View all devices and analytics	-	Opt.	Yes	Yes	Read	-	-
Add and see own devices; request access	-	-	Yes	Yes	Yes	Yes	One device

Opt. = only when the install grants super admins implicit tenant-admin rights, or the super admin is assigned a role in that tenant. A dash means no right by that role alone; a person can hold several roles. Approvers cannot approve their own requests; changes to prod can require a second approver.

What is audited

- Every state-changing API call (who, what, before/after, source IP, user agent, request id).
- Login, MFA, role changes (including Owner transfer and super-admin changes), API-key creation and use.
- Policy versions: created, tested, applied, rolled back.
- Access requests, approvals, grants, expiries, revocations; share links created, used, expired.
- Device lifecycle seen via Headscale: registered, expired, removed, tags changed, routes approved.

Integrity and export

- **Hash chain:** each event stores the hash of the previous one per tenant, so deletion or edits are detectable. Periodically publish the chain head to object storage or a transparency log.
- **Append-only:** the audit store accepts inserts only; nobody can edit or delete events, not even the Owner.
- **Export:** filtered CSV/JSONL download, scheduled export to S3-compatible storage, streaming to syslog, webhook or a SIEM.
- **Retention:** configurable; default 400 days, longer for compliance setups.
- **Limit to state clearly:** this is a control-plane audit. It records who changed access, not which packets flowed between devices.

10. Access requests and time-limited access

This is the feature that turns a VPN into an access-management product. Users ask, approvers decide, the system grants for a limited time and always cleans up.

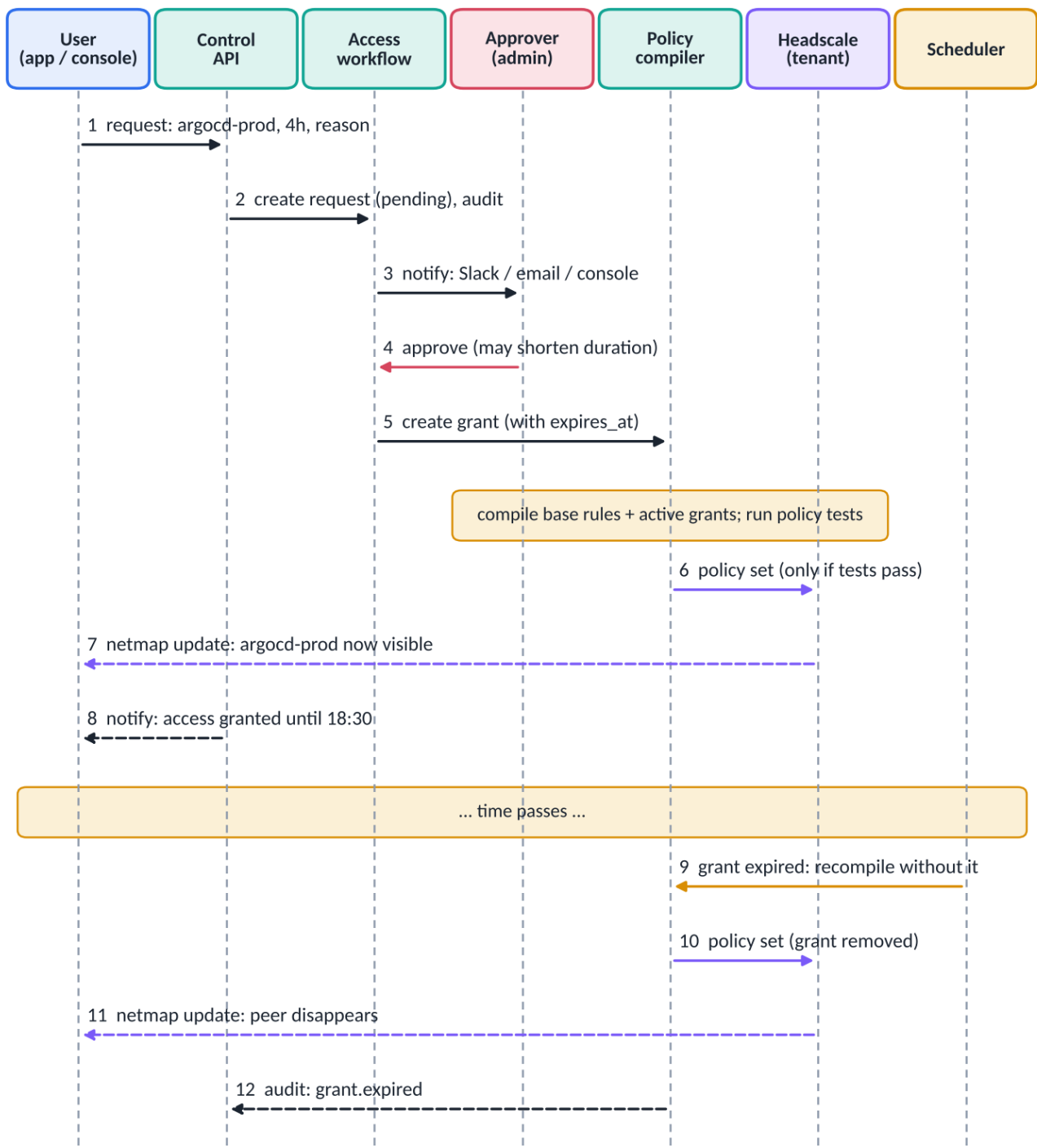


Figure 7. Access request, approval, grant and automatic expiry.

Design rules

- **Request fields:** resource or environment, duration (bounded by tenant policy), reason, optional ticket link.
- **Approval policy per environment:** dev can auto-approve for a few hours; uat needs any admin; prod needs a named approver and a reason; self-approval never.
- **Grant = row with expires_at.** Only active grants are compiled into policy. The reaper runs every 30 seconds and recompiles on any expiry.
- **Extension and early revoke:** both are first-class actions, audited like everything else.
- **Break-glass:** tenant admins can grant immediately with a mandatory reason; it triggers an alert to the security channel.
- **Notifications:** console, email, Slack/Teams/Telegram, with approve/deny buttons where the channel supports them.

Behaviour to test before promising it

When a rule is removed, are already-open connections cut immediately or only new ones blocked? Tests should cover TCP sessions, SSH and long-lived streams. For SSH, also consider Headscale's SSH policy with check actions to force periodic re-authentication [verify].

11. Guest and share links

Headscale does not offer Tailscale-style cross-tailnet node sharing [verify], so guests join the tenant as tightly restricted users. The link is the invitation; the grant is the leash.

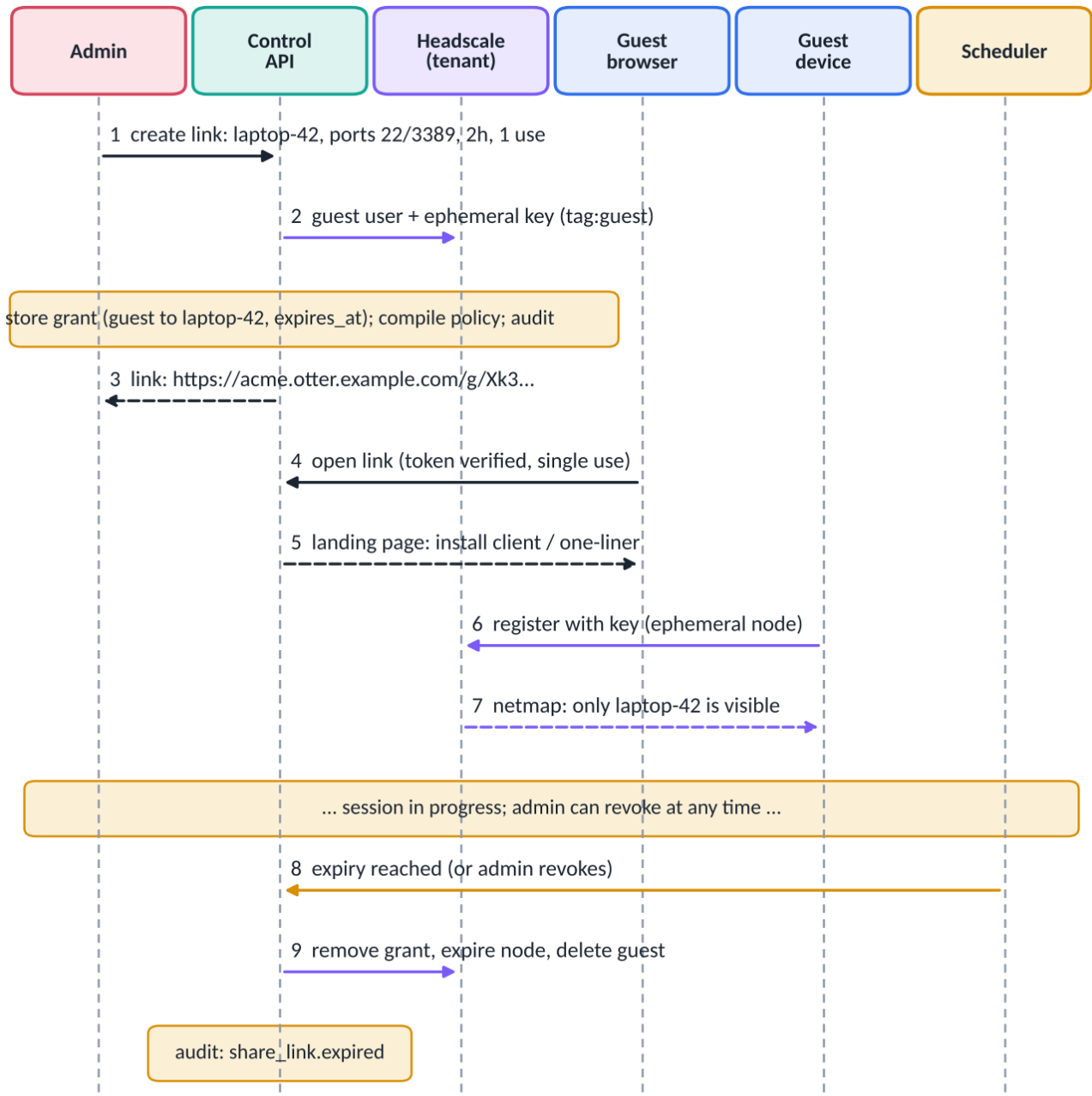


Figure 8. Share link for one device: create, redeem, restrict, expire.

Controls

- **Single use or limited uses**, with the token stored hashed and compared in constant time.
- **Narrow scope:** one device, listed ports only, no subnet routes, no exit nodes, no DNS names of other devices.
- **Ephemeral nodes** tagged `tag: guest`; removed automatically soon after going offline or at expiry.

- **Visible and revocable** in the console: who created it, whether it was redeemed, a live session indicator, a revoke button.
- **Rate-limited redemption page** with optional email or code verification of the guest.
- **Later phase:** browser-based access (web SSH/RDP gateway, as headscale-console demonstrates) so guests need not install anything.

12. Device onboarding, CLI and agent

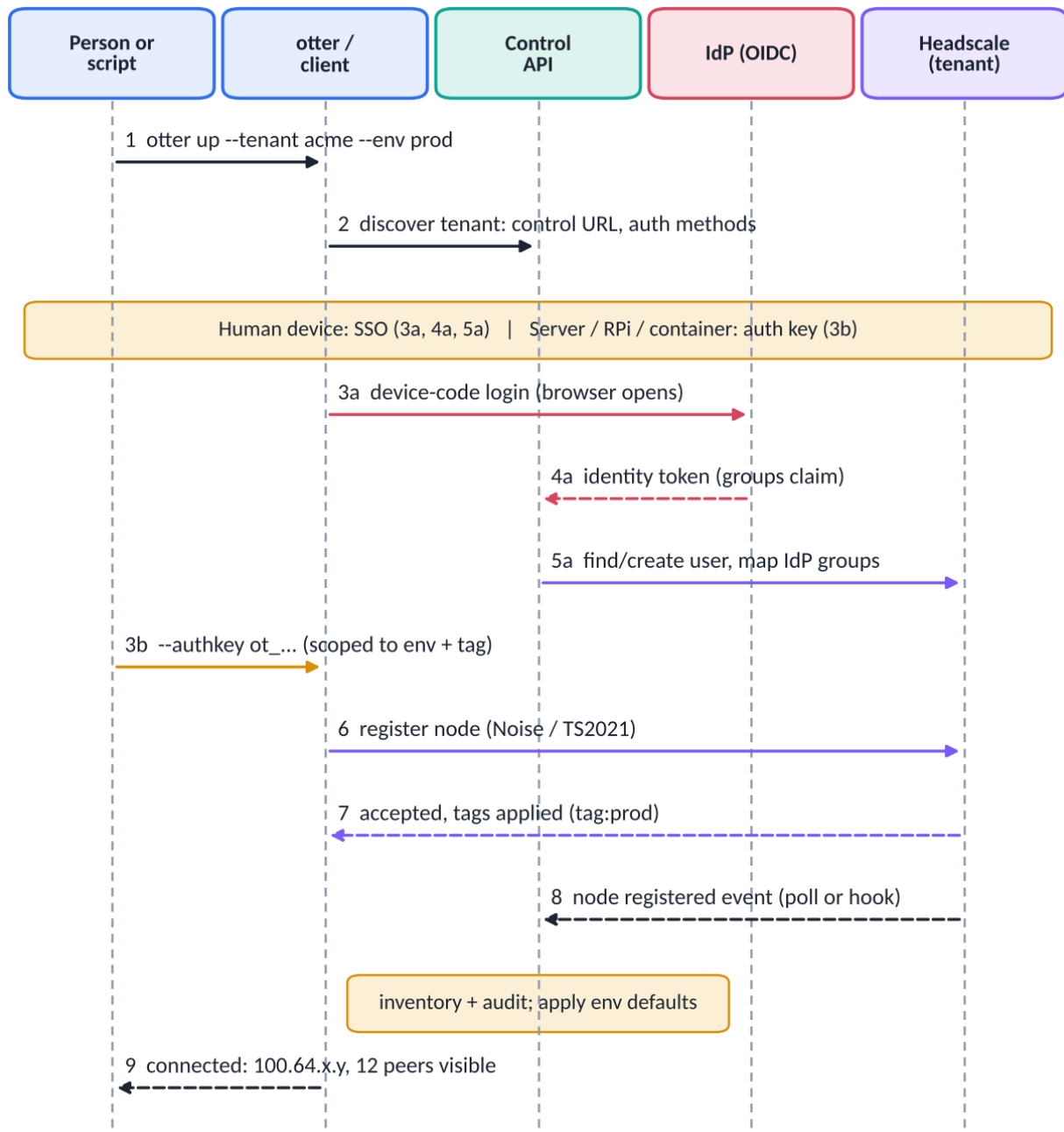


Figure 9. Onboarding a human device with SSO, or a server / RPi / container with a scoped auth key.

Recommended method per device type

Device	Client	Authentication	Result
Laptop / desktop	Official Tailscale app	SSO (device code in browser)	User-owned node, group by IdP claims
Phone / tablet	Official Tailscale app	SSO or QR from the console	User-owned node
Server (VM, bare metal)	Official tailscaled + otter helper	Scoped, reusable auth key	Tagged node (e.g. tag:prod)
Raspberry Pi / IoT	Official tailscaled + otter helper (ARM)	Scoped auth key, one-liner install	Tagged node, systemd service
CI runner / container	Sidecar or embedded SDK	Ephemeral, single-use key	Ephemeral tagged node
Kubernetes workload	Operator-managed proxy	Operator mints per-resource keys	Tagged proxy or service node

The otter command line

A single static Go binary that installs, configures and drives the official Tailscale client (`tailscaled`) and talks to the Otterscale API, with a daemon mode for servers. It is a helper around the official client, not a replacement for it.

```
# install on a server or Raspberry Pi
curl -fsSL https://acme.otter.example.com/install.sh | sh

# join as a tagged server (non-interactive)
otter up --tenant acme --env prod --authkey ot_live_9f2... --hostname api-prod-01

# join as a person (opens a browser to sign in with SSO)
otter up --tenant acme

otter status                # peers, direct vs relayed, versions
otter accounts list         # all tenants/accounts on this machine
otter accounts switch contractor-xyz
otter envs list             # prod, uat, dev you can see
otter request prod/argocd --for 4h --reason "hotfix 1234"
otter share create laptop-42 --ports 22 --for 2h --uses 1
otter ssh deploy@api-prod-01 # via mesh, honours policy
otter down | logout | doctor # doctor checks DNS, DERP, clock skew, proxy issues
```

Agent mode

- Heartbeat with OS, client version, uptime and optional posture facts (disk encryption, OS patch level) for analytics and policy later.
- Auto-update within the compatibility window of the tenant's Headscale version.
- Optional connection sampling for flow-style analytics, off by default and clearly disclosed.

13. SDKs, Terraform and Kubernetes (the VPC-like layer)

Deliverable	Purpose	Notes
Control API (REST + gRPC, OpenAPI)	Single source of truth for all clients	Versioned; scoped keys; idempotency keys on writes.
Terraform / OpenTofu provider	Tenants, environments, groups, rules, auth keys as code	Same mental model as cloud VPC resources.
Go SDK	Programmatic control + embeddable node library	Embedded node uses the open-source userspace network library, so a service can join a tenant without root or a TUN device.
Python and TypeScript SDKs	Automation, internal tools, bots	Generated from OpenAPI.
Kubernetes operator + CRDs	Expose services, egress, subnet routers, API server access	Talks to your API (section below).
Helm chart and Docker images	Run the platform itself	Section 16.

Terraform example

```
resource "otterscale_environment" "prod" {
  tenant      = "acme"
  name        = "prod"
  strict_isolation = true           # dedicated Headscale instance
}

resource "otterscale_group" "backend" {
  tenant = "acme"
  name   = "backend"
  members = ["bob@acme.com", "carol@acme.com"]
}

resource "otterscale_access_rule" "backend_dev" {
  environment = "dev"
  group       = otterscale_group.backend.name
  ports       = ["*"]
}

resource "otterscale_authkey" "argocd_prod" {
  environment = otterscale_environment.prod.name
  tags        = ["tag:argocd"]
  reusable    = true
  expiry      = "720h"
}
```

Kubernetes and Argo CD

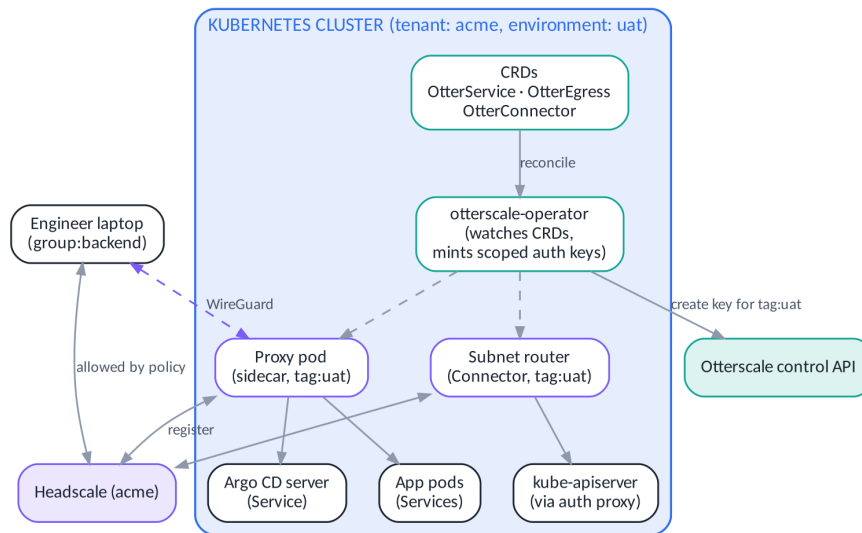


Figure 10. Operator-managed access to services and the API server inside a cluster, scoped to one environment.

Why your own operator

Tailscale's operator authenticates through OAuth to Tailscale's own control plane, and Headscale compatibility remains an open request upstream, with at least one third-party packaging project claiming support. Rather than bet on that, the Otterscale operator talks to *your* control API, which already knows tenants, environments and scoped keys. Pin the proxy engine version to what your Headscale supports.

```
apiVersion: otterscale.example.com/v1
kind: OtterService
metadata:
  name: argocd-server
  namespace: argocd
spec:
  tenant: acme
  environment: uat           # becomes tag:uat on the proxy node
  hostname: argocd-uat      # MagicDNS name inside the tenant
  target:
    service: argocd-server
    port: 443
```

Argo CD patterns

- **People reach Argo's UI/API:** publish argocd - server as a OtterService in the right environment. Group rules decide who sees it; prod access can be request-only.
- **Argo reaches private clusters:** run a subnet router (OtterConnector) in each target cluster or network, advertise only the API server route, and approve it via policy.
- **CI pipelines:** ephemeral, single-use auth keys per job, tagged tag : ci with minimal rules.

14. Analytics

Insight	Source	Feasible?
Device inventory, OS and client versions, online/offline, last seen	Headscale API + agent heartbeat	Yes
Adoption: active users, new devices per week, onboarding funnel	Control DB + audit	Yes
Access workflow: request volume, approval time, most requested resources	Control DB	Yes
Key and node expiry forecast; stale devices; users without devices	Headscale API	Yes
Policy health: unused rules, over-broad rules, groups without members	Policy compiler + lint	Yes
Direct versus relayed connections, DERP latency by region	Agent reports and relay metrics	Yes, with agent
Platform health: Headscale up, reload latency, API errors, tenant counts	Prometheus metrics	Yes
Who talked to whom, bytes, ports (flow logs)	Not provided by Headscale	Only with an optional sampling agent; privacy review needed [verify]

Show analytics at two levels: a tenant dashboard (health, adoption, access activity) and a platform dashboard for operators (tenant count, upgrade status, resource use). Keep raw events for a limited period and aggregate for long-term trends.

15. Clients: the official Tailscale apps

Decision: Otterscale does not ship its own client app. People and devices use the unmodified, official Tailscale clients, pointed at the tenant's Headscale server. Otterscale adds a **Connect** area to the web console that makes this easy, and the otter helper for servers. This removes the most expensive part of the original plan (signing, store review, Apple rules, security updates for a client) and keeps adoption simple.

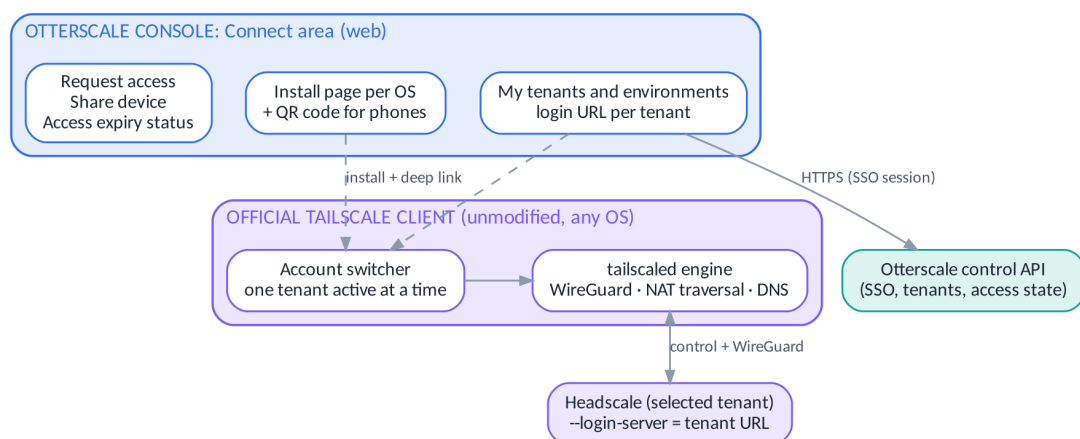


Figure 11. Client model: the console guides people; the unmodified Tailscale app connects to the selected tenant's Headscale.

How people connect

- **Install:** the Connect page detects the OS and links to the official download or store page.
- **Point the app at the tenant:** the page shows the tenant's login URL and the steps for that OS (see the table below). Sign-in is your SSO.
- **Several tenants:** the Connect page lists every tenant and environment the person belongs to, each with its own login link. In the Tailscale app they use its account switcher; **one tenant is connected at a time**.

- **Expiry and requests:** access is enforced by policy in the network, so time-limited access works regardless of the client. Requests, approvals and sharing happen in the web console, with email or chat notifications.

Pointing each official app at your server

Platform	How	Notes
Linux, BSD, Raspberry Pi, servers	<code>tailscale up --login-server <tenant URL></code> , optionally with a scoped auth key	Documented path; <code>otter up</code> wraps it.
Windows	Command line or a policy/registry setting for the control URL	Exact method varies by client version [verify].
macOS	A setting or configuration profile for the control URL	Exact method varies by client version [verify].
Android	Choose an alternate coordination server in the app before signing in	Location of the option varies by version [verify].
iOS	Alternate coordination server setting in the app	Less discoverable; test on each release [verify].

Because these steps differ by version, the Connect page generates the right instructions per OS, and each release is checked by hand on every platform (section 20). For company-managed devices, MDM profile templates (for example Intune or Jamf) can preconfigure the control URL where the client supports it [verify].

What you give up, and the workaround

Without a custom client	Workaround
Tailscale's name and logo appear in the app	Say so in the docs; use neutral wording ("compatible with Tailscale clients"). See the trademark note in section 20.
No in-app request-access button	Request and approve in the web console; deep links and chat or email notifications.
No tenant list inside the app	The console's Connect page lists tenants with a login link each.
App shows a generic state when access expires	Console shows access status and warns before expiry; the device simply loses reachability.
Client updates are the user's or the MDM's job	Console shows each device's client version and flags ones outside Headscale's supported window.

Risks to plan for

- **Compatibility window:** v0.29 requires client v1.80.0 or newer and targets roughly the last ten releases, so old devices need an upgrade nudge.
- **Upstream changes:** the official apps can change behaviour (including how custom servers are set) without notice; keep per-OS test notes and re-check each release.
- **Vendor dependence:** you rely on Tailscale continuing to allow custom control servers in its apps. Today this is widely used with Headscale, but it is not a guarantee [verify current policy].
- **Branding:** users see Tailscale branding; keep your own brand in the console and docs.

When to revisit building your own client

Only if real usage shows the official apps are a blocker, for example people repeatedly failing at tenant switching, a need for simultaneous connections to several tenants, or an Apple/Google policy change. The earlier design for a custom client (desktop shell, then Android, then iOS) is not part of this plan.

16. Deployment and packaging

Yes, ship Docker images. The platform is a small set of images; the topology on the next page shows where each runs and which parts need UDP.

Image	Contents	Notes
otterscale-platform	API, worker, orchestrator in one binary with subcommands; console assets	Stateless; scale horizontally; needs Postgres.
otterscale-headscale	Pinned Headscale + Litestream + health probe + endpoint that renders per-tenant config	One container per tenant, SQLite on a persistent volume.
otterscale-allinone	Platform + reverse proxy + one or more pinned Headscale binaries + Litestream; supervisor spawns one Headscale process per tenant	Single container for N tenants; needs one persistent volume (section 16).
otterscale-derp	DERP + STUN relay	Needs UDP 3478 and TLS; run on a VM per region.
otterscale-operator	Kubernetes operator	Plus a Helm chart and CRDs.
otter	CLI/agent binaries and install script	Linux, macOS, Windows, ARM.

Where each part can run

Target	Control platform	Per-tenant Headscale	DERP / STUN	Verdict
Single VM + Docker Compose	Yes	Yes (Docker socket)	Yes (same or second VM)	Best start for small teams and the quickstart
AWS: ECS or EC2 + RDS + EC2 for DERP	Yes	ECS services or EC2	EC2 with UDP security group	Good for production SaaS
Kubernetes + Helm	Yes	StatefulSet per tenant	DaemonSet or VMs with host ports / UDP LB	Best for multi-tenant at scale
Railway	Yes (HTTP)	Yes via all-in-one (processes inside one service; needs a volume [verify])	No: no inbound UDP, TCP proxy only	Multi-tenant control plane possible; DERP must live on a VM
Render and similar PaaS	Likely yes	Likely yes via all-in-one [verify volumes]	Check UDP support [verify]	Treat like Railway until verified

A PaaS can host the control platform, and with the all-in-one image it can also host many tenants as processes inside one service, but it cannot run DERP if it lacks UDP. In the early days you can point a tenant at public relay infrastructure to get started, but that creates a dependency on someone else's relays; plan to run your own for production.

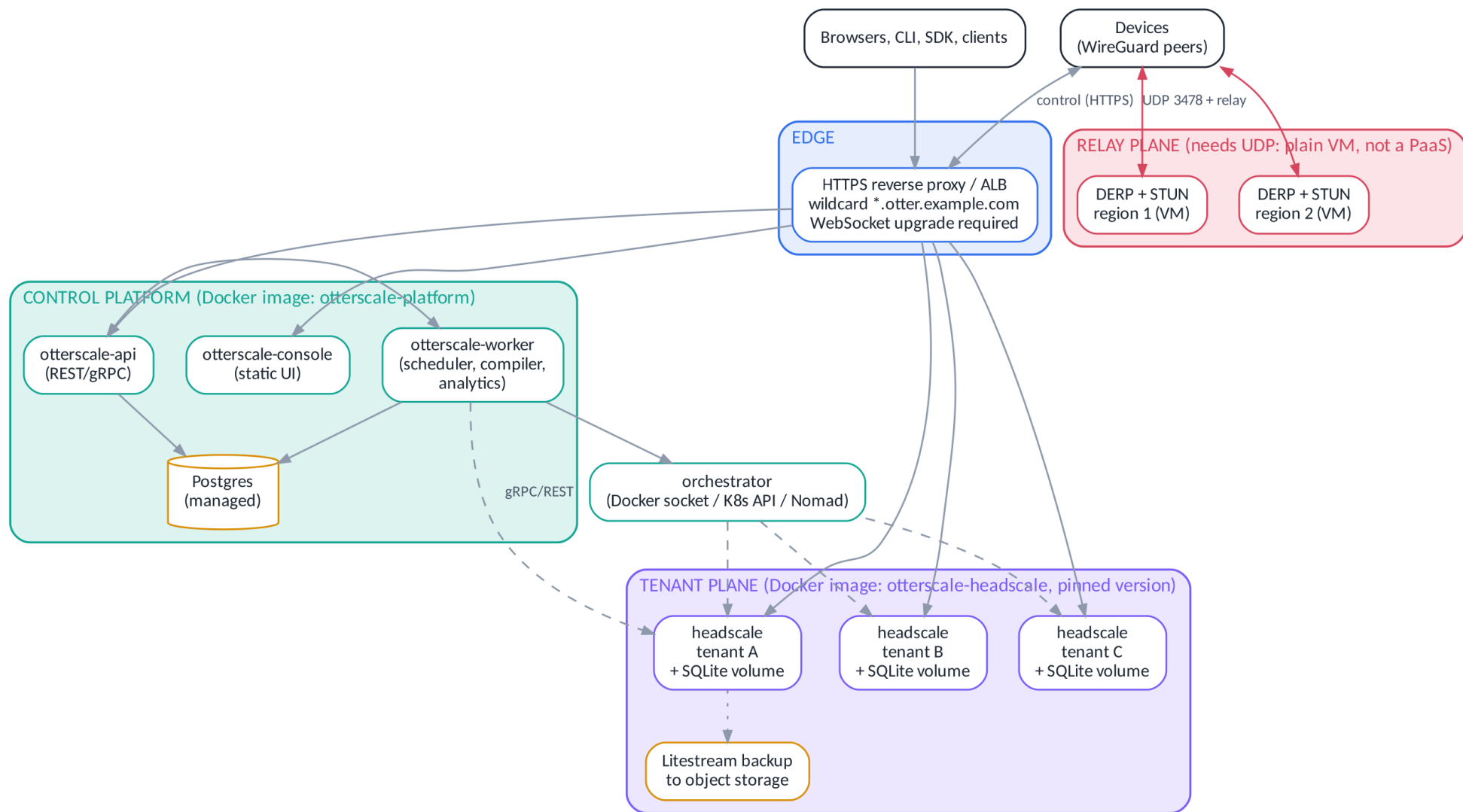


Figure 12. Deployment topology. Edge and control platform can be PaaS or containers; the relay plane needs real UDP; tenant instances need a place to be orchestrated.

Single-container mode (all-in-one)

One image can run any number of tenants. It does **not** run one Headscale process for many tenants (Headscale serves one tailnet per instance); it runs **one Headscale process per tenant** under a supervisor, all inside one container. Creating a tenant from the Platform area of the console starts a new child process, with no new container and no restart of the others.

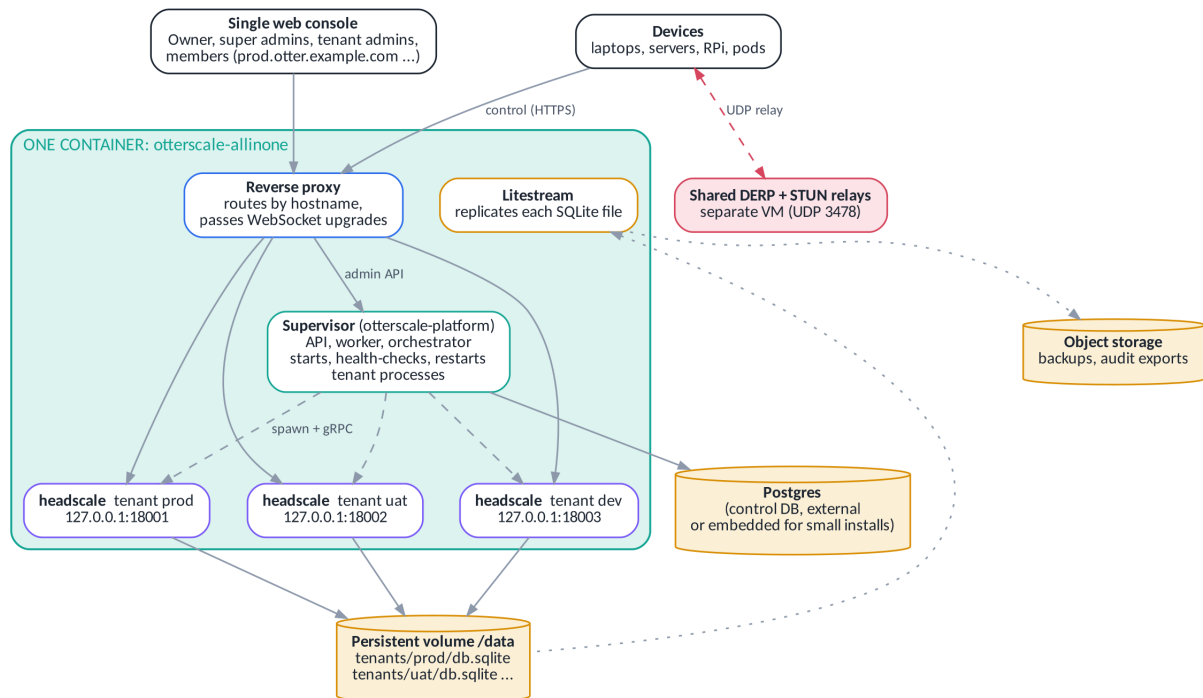


Figure 13. All-in-one mode: a supervisor, a hostname-routing proxy and one Headscale process per tenant, with a shared relay outside the container.

How it behaves

- **Add a tenant:** create the data directory, render the tenant config, pick a free port from the reserved range, start the process, add the proxy route, record it.
- **Config per tenant:** own listen, gRPC and metrics ports, own SQLite file, own IP prefix; the embedded DERP/STUN server is disabled so tenants do not fight over UDP 3478. All tenants use the shared relay fleet.
- **Versions:** the image can carry several Headscale binaries side by side, so tenants move through the required upgrade path one minor version at a time, canary first.
- **Recovery:** the supervisor restarts crashed children and reports health to the Platform area of the console; Litestream replicates each SQLite file to object storage.
- **Isolation inside the container:** run each child as its own OS user with its own directory permissions and resource limits; this is weaker than separate containers because the kernel and memory are shared.

Orchestrator drivers

Driver	Runs tenants as	Isolation and limits	Best for
process (all-in-one)	Child processes in one container	Process, DB, port; OS-level limits; shared fate if the container restarts	Quickstart, homelab, small startups, single PaaS service
docker	One container per tenant on one or more hosts	Per-container CPU and memory limits	Medium installs on VMs
kubernetes	A StatefulSet or Deployment per tenant	Strong; schedulable across nodes	Hosted service at scale

Limits to plan for

One container means one machine and shared fate: restarting it interrupts the control plane of every tenant on it (devices that are already connected generally keep working, but new logins and policy changes pause [verify]). Measure how many tenants a host holds before promising a number, keep per-tenant quotas, and move large tenants to their own container or pod. Because a tenant is already an isolated process with its own directory, moving it to another driver is a copy of that directory plus a routing change.

Illustrative settings: ORCHESTRATOR=process, TENANT_PORT_RANGE=18000-18999, DATA_DIR=/data, HEADSCALE_BIN_DIR=/opt/headscale, RELAY_MAP_URL=https://derp.example.com/map.json, PUBLIC_BASE_DOMAIN=otter.example.com.

Quickstart Compose sketch

```
services:
  postgres: { image: postgres:17, volumes: [pg:/var/lib/postgresql/data] }
  platform:
    image: ghcr.io/example/otterscale-platform:0.1
    environment:
      DATABASE_URL: postgres://otterscale@postgres/otterscale
      PUBLIC_BASE_DOMAIN: otter.example.com
      ORCHESTRATOR: docker # docker | kubernetes
    volumes: ["/var/run/docker.sock:/var/run/docker.sock"] # restrict via socket proxy in production
    depends_on: [postgres]
  caddy:
    image: caddy:2 # wildcard TLS, WebSocket upgrade enabled
    ports: ["80:80", "443:443"]
  derp:
    image: ghcr.io/example/otterscale-derp:0.1
    ports: ["3478:3478/udp", "8443:8443"]
volumes: { pg: {} }
```

Operations

- **Upgrades:** honour Headscale's strict upgrade path; upgrade a canary tenant first; keep a rollback image and SQLite snapshot.
- **Backups:** continuous SQLite replication per tenant, nightly Postgres dumps, test restores monthly.
- **High availability:** a Headscale instance is a single process [verify]; mitigate with fast restarts, health checks and per-tenant isolation. Established device connections generally keep working while the control server restarts, but new logins and policy changes pause.
- **Observability:** Prometheus metrics, structured logs with tenant and request ids, dashboards for reload time and API latency.
- **Reverse proxy:** publish a tested reference config because proxy mistakes are the most common Headscale support issue.

17. Security and threat model

The control plane decides who can reach what, so its compromise is network compromise. Treat it like an identity provider.

Threat	Mitigation
Stolen admin session or API key	OIDC with MFA, short sessions, step-up auth for risky actions, scoped and expiring API keys, IP allow-lists optional.
Compromised Headscale API key stored by the platform	Per-tenant keys, envelope-encrypted with a KMS, never exposed to the browser; Headscale API reachable only on a private network.
Cross-tenant data leakage	Tenant id on every row and enforced in the data layer; instance-per-tenant; automated isolation tests in CI.
Policy mistake locks everyone out or opens too much	Generated tests, diff preview, staged apply, auto-rollback, break-glass path for the Owner.
Share link abuse	Hashed single-use tokens, narrow rules, short expiry, rate limits, revocation, alerts on unusual redemption.
Tampered audit log	Append-only store, hash chain, external anchoring of chain heads, restricted DB roles.
Malicious client binary or update	Signed releases, reproducible builds where possible, SBOMs, update signature verification.
Dependency or upstream vulnerabilities	Renovate-style updates, vulnerability scanning, pinned versions with fast-follow policy, security contact in SECURITY.md.
Denial of service on coordination	Rate limiting at edge, per-tenant resource limits, isolated instances.
Owner or super admin account compromise or misuse	Mandatory MFA with hardware keys, step-up MFA for the Platform area, only the Owner can add super admins, no standing access to tenant contents in hosted setups, consented time-boxed support access, two-person rule for destructive actions, platform actions mirrored into the tenant audit log.
Privilege escalation through the single console	Server-side authorization on every API call, roles read from the database (not the token), per-role API tests in CI, deny by default, hidden UI treated as cosmetic only.
Setup link claimed by an attacker before the Owner exists	One-time token in host logs only, short expiry, regeneration only from the host shell, setup mode permanently off afterwards, health warning while on.
Noisy or malicious tenant in a shared container	Per-process OS user, resource limits (cgroups/ulimits), per-tenant rate limits, quota enforcement; move heavy tenants to their own container or pod.
Orchestrator abuse (Docker socket access)	Socket proxy with minimal verbs, or Kubernetes RBAC limited to a dedicated namespace.

Before v1.0: an external security review of the control plane, tenant isolation and client updater; a public disclosure policy; and a signed release process.

18. Roadmap

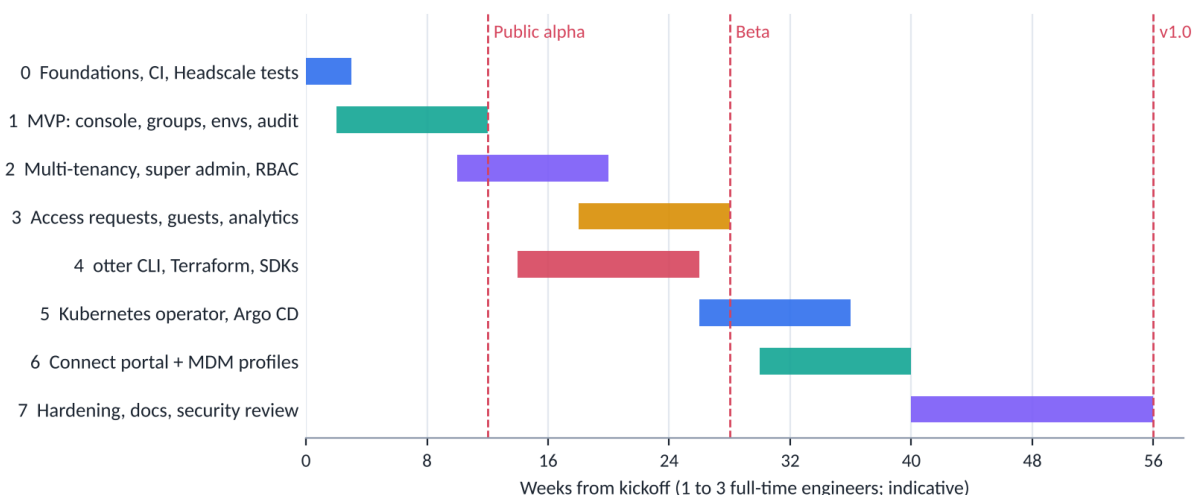


Figure 14. Indicative plan for one to three full-time engineers. Phases overlap; dates are estimates, not commitments.

Phase	Exit criteria
0 Foundations	Repo, CI with Headscale integration tests against several client versions, API client generated, reference reverse-proxy config.
1 MVP (public alpha)	One tenant: SSO login, devices, groups, environments, compiled policy with tests, onboarding wizard, audit log, Docker Compose quickstart.
2 Multi-tenancy and role-based console	First-run setup creates the single Owner; Owner adds super admins; super admins create, suspend, upgrade, back up and delete tenants and assign tenant admins end to end (process driver first); tenant subdomains; platform and tenant RBAC with per-role API tests; isolation tests green.
3 Access workflows (beta)	Requests, approvals, expiring grants, guest links, notifications, analytics v1; reaper proven under failure injection.
4 CLI and SDKs	otter on Linux, macOS, Windows and ARM; Terraform provider; Go, Python, TypeScript SDKs published.
5 Kubernetes	Operator, Helm chart, Argo CD guide, subnet-router recipe; tested on at least two distributions.
6 Connect area	Install page per OS, per-tenant login links and copy-paste commands, QR to the setup page, MDM profile templates, client-version warnings, access-expiry status.
7 Hardening	External security review, docs, upgrade tooling, public v1.0 release.

19. Is this good for open source? Deep analysis

Landscape

Project	What it is	Overlap with Otterscale
Headscale	Open-source Tailscale control server (the engine you build on)	Dependency, not competitor. Ecosystem partner.
Headplane and other Headscale UIs	Community web UIs. Headplane covers machines, ACLs and tags, OIDC and DNS settings	Direct overlap on admin CRUD. Your edge is tenants, environments, workflows, audit, CLI, SDK and clients.
NetBird	Fully open-source mesh VPN with its own clients, management server, web UI, SSO and DNS	Closest full-stack competitor. Different protocol and clients: not compatible with Tailscale clients.
Netmaker	WireGuard mesh with site-to-site routing and a web UI	Different architecture; overlaps on use cases.
Tailscale (SaaS)	Proprietary control plane; market leader	The reference experience; also your best marketing contrast: self-hosted and open.
ZeroTier, Nebula, Firezone, Twingate and others	Various overlays and zero-trust access products [from general knowledge, verify current status]	Adjacent; useful for positioning.

Why it can work

- **Clear unmet need:** small teams want Tailscale-like ease without per-seat pricing or SaaS dependence, but Headscale alone means CLI work and hand-written policy.
- **Compatibility moat:** being able to use existing Tailscale-protocol clients removes the hardest adoption barrier.
- **Workflow differentiation:** time-boxed access with approvals and audit is what security-minded startups pay for, and it is rare in self-hosted tools.
- **Tenancy fills a structural gap:** Headscale's single-tailnet model leaves MSPs, agencies and platform teams with no good option.
- **Strong infrastructure-community fit:** homelab, self-hosting and DevOps audiences actively adopt and contribute to tools like this.

What can go wrong

Risk	Likelihood	Impact	Mitigation
Headscale or client protocol changes break you	Medium	High	Adapter layer, pinned versions, compatibility CI, contribute upstream
Scope explosion (portal, SDKs, operator at once)	High	High	Strict phasing; no custom client apps
Security incident in the control plane	Low-medium	Very high	Security review, minimal privileges, bug-bounty-style disclosure
Headplane or Headscale adds the same features	Medium	Medium	Move fast on workflows and tenancy; keep integration friendly, consider upstreaming generic parts
Trademark or legal friction with Tailscale	Low	High	Independent name and branding; neutral wording such as "compatible with Tailscale clients"; legal check
Maintainer burnout, no funding	Medium-high	High	Revenue model, co-maintainers, clear contribution guide
Support load from reverse-proxy and NAT issues	High	Medium	otter doctor, reference configs, documented troubleshooting

Licensing and governance

- **Core platform:** AGPL-3.0 protects against closed hosted forks while keeping the code fully open. If you prefer maximum adoption and are comfortable with cloud providers reselling it, Apache-2.0 is the alternative.
- **SDKs, CLI, operator and Terraform provider:** Apache-2.0, so they can be embedded in any company's tooling.
- **Contributions:** use a DCO or a CLA, decide early; publish a code of conduct, SECURITY.md and a public roadmap.
- **Trademark policy:** register your own name, document acceptable use for forks and hosted offerings. Otterscale evokes otters (teamwork, tight-knit groups); its "-scale" ending signals the Headscale ecosystem but sits close to the Tailscale mark, so keep branding clearly independent and say "compatible with Tailscale clients" instead of using their marks.
- **Upstream relationship:** engage with the Headscale maintainers early; feature gaps (events, policy hooks, tenancy primitives) are better solved together.

Sustainability options

Model	How it works	Fit
Managed hosting	You operate tenants and relays for customers	Strong: orchestration is already your core
Open-core enterprise features	SCIM, SIEM streaming, long audit retention, advanced approval chains, SSO policy	Good; keep security basics open
Support and services	Setup, migration, SLAs	Good early revenue
Sponsorship and grants	GitHub Sponsors, foundation or infrastructure grants	Supplement, not a plan

Go / no-go

Go, with these conditions: (1) the first release is console + policy engine + audit + CLI, not apps; (2) tenancy and access workflows are the headline features, because admin CRUD alone competes with Headplane; (3) compatibility CI against Headscale and Tailscale clients exists from day one; (4) you commit to a security review before calling it 1.0. **Re-evaluate** if after the public alpha fewer than a few dozen real teams adopt it, or if Headscale upstream ships comparable multi-tenancy and approvals.

20. Open questions and verification checklist

I could not confirm these from sources; each should become an early spike or integration test.

Question	How to resolve	Affects
Do devices truly hide peers that policy does not allow, in Headscale's network map?	Integration test with three nodes and group rules	Group visibility promise (sec. 6)
Are established connections terminated when a rule is removed?	Test TCP, SSH and long streams across policy reload	Time-limited access guarantees (sec. 10)
Does the official client switch reliably between several Headscale accounts, including re-login and key expiry, on each OS?	Test profile switching on current Windows, macOS, Linux, Android and iOS builds against a pinned Headscale	Connect area (sec. 15)
Which policy and node operations does the v2 API cover, and can you set policy through it?	Read API reference; script against a test instance	Compiler adapter, SDK reuse
Does Headscale emit events or hooks, or is polling required?	Check the current release; plan an upstream contribution if not	Real-time UI, audit of device events
Exactly how each official app is pointed at a custom control server (Windows, macOS, iOS, Android), and whether MDM can preconfigure it	Test on current app versions; document per OS and re-check each release	Connect area (sec. 15)
Headscale support for node sharing between tailnets	Check docs and issues	Guest design (sec. 11)
UDP support on Render and other PaaS targets	Read their networking docs	Deployment guidance (sec. 16)
How many tenant processes fit in one container or host, and what is the memory and CPU cost per idle and active tenant?	Benchmark with synthetic tenants and devices	All-in-one capacity guidance (sec. 16)
Is the name Otterscale (and the command name <code>otter</code>) clear? Check domain, GitHub organisation, npm, PyPI, crates.io, Docker Hub, Homebrew/apt command collisions and trademarks, including confusion with Tailscale and Headscale.	Registry and trademark searches (India, WIPO, US/EU as needed) and a short legal review	Branding and launch
Does the chosen PaaS give persistent volumes and enough internal ports for the all-in-one image?	Read platform docs; run a trial deployment	PaaS guidance (sec. 16)
How far can super admin power over tenant policy be limited (for example tenant-approved changes)?	Design review; consider tenant-held approval keys	Trust model for hosted tenants (sec. 7)

Question	How to resolve	Affects
Headscale high-availability options	Read docs and issues; test restart behaviour under load	SLA claims for hosted tenants
Whether the Kubernetes operator from Tailscale actually works against Headscale in your version combination	Spike on a test cluster	Operator scope (sec. 13)

Appendix A: control API sketch

Area	Endpoints (illustrative)
Tenants	POST /v1/tenants, GET /v1/tenants/{t}, PATCH, DELETE; POST /v1/tenants/{t}/upgrade
Environments	GET/POST /v1/tenants/{t}/environments; PATCH /v1/environments/{e}
Members and groups	GET/POST /v1/tenants/{t}/members; PUT /v1/members/{m}/role; GET/POST /v1/tenants/{t}/groups; PUT /v1/groups/{g}/members
Access rules and policy	GET/POST /v1/environments/{e}/rules; POST /v1/tenants/{t}/policy:preview; POST /v1/tenants/{t}/policy:apply; POST /v1/tenants/{t}/policy:rollback
Devices	GET /v1/tenants/{t}/devices; PATCH /v1/devices/{d} (name, tags, notes); POST /v1/devices/{d}:expire; DELETE
Auth keys	POST /v1/environments/{e}/authkeys (tags, reusable, ephemeral, expiry); DELETE /v1/authkeys/{k}
Access workflow	POST /v1/access-requests; POST /v1/access-requests/{r}:approve :deny; POST /v1/access-grants/{g}:revoke :extend
Share links	POST /v1/share-links; GET /g/{token} (public landing); POST /v1/share-links/{l}:revoke
Audit and analytics	GET /v1/tenants/{t}/audit?actor=&action;=&from;=&to;=; POST /v1/tenants/{t}/audit:export; GET /v1/tenants/{t}/analytics/{report}
Events	GET /v1/tenants/{t}/events (Server-Sent Events)
Platform setup and admins	POST /platform/v1/setup (only while no Owner exists); GET/POST /platform/v1/admins (Owner only); POST /platform/v1/ownership:transfer :accept; DELETE /platform/v1/admins/{a} (Owner only)
Platform (Owner and super admin scope)	GET/POST /platform/v1/tenants; PUT /platform/v1/tenants/{t}/admins (assign tenant admins); POST /platform/v1/tenants/{t}:suspend :resume :delete :upgrade; GET /platform/v1/capacity; GET /platform/v1/relays; POST /platform/v1/tenants/{t}/support-session (time-boxed, consented); GET /platform/v1/audit

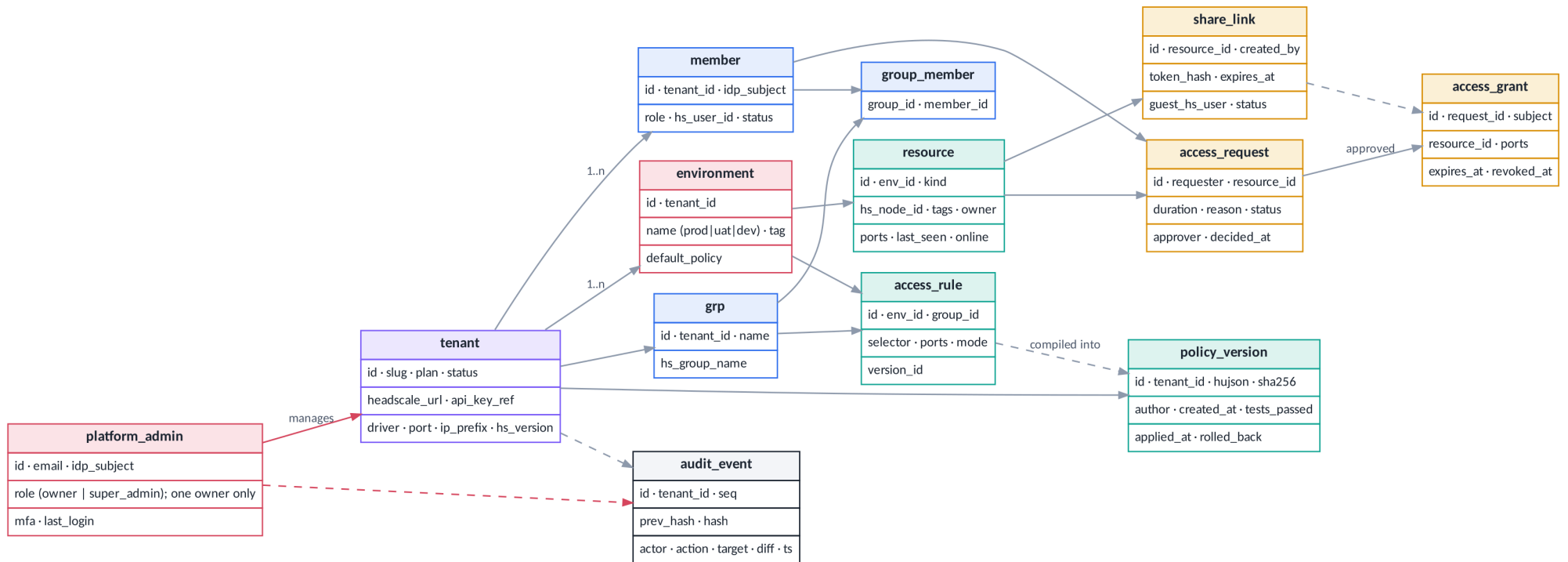


Figure 15. Control-plane data model (core tables). Amber tables implement access requests, grants and share links; the audit table is append-only.

Appendix B: repository layout

```
otterscale/  
  cmd/           platform, worker, orchestrator, otter, operator  
  internal/  
    headscale/   adapter around the Headscale API (the only package that talks to it)  
    policy/      intent model, compiler, linter, test generator  
    access/      requests, grants, share links, reaper  
    audit/       append-only store, hash chain, exporters  
    tenancy/     orchestrator drivers: process (all-in-one), docker, kubernetes  
    platform/    setup, Owner and super admins, tenant lifecycle, plans, quotas, support sessions  
    authz/       role model, permission checks, per-role test fixtures  
    analytics/   collectors and report queries  
  web/          single role-based console (TypeScript): areas platform/, workspace/, network/, my-access/  
  connect/      install pages, per-OS guides, MDM profile templates  
  sdk/          go, python, typescript, terraform-provider  
  deploy/       compose, helm, reverse-proxy references, derp  
  docs/         architecture decisions, runbooks, API reference  
  test/        integration harness: Headscale + multiple client versions
```

Appendix C: sources checked

- Headscale v0.29.3 package page (pkg.go.dev), published 29 Jul 2026; Headscale releases page (github.com/juanfont/headscale/releases), including v0.29.0 notes on grants, policy tests, SSH check and minimum client v1.80.0.
- Headscale documentation mirror: web UI integrations list; FAQ (scaling, SQLite, containers and reverse proxies); client and OS support; ACL reference.
- Headscale documentation index on docsearch.algolia.com: API v2 compatibility with the Tailscale API, supported client policy, community tools.
- Infralovers, "Headscale: A Self-hosted Tailscale Alternative" (May 2026); wz-it.com Headscale overview (mid 2026).
- Headplane repository (github.com/tale/headplane); Pinggy, "Top Open Source Tailscale Alternatives in 2026" (23 May 2026).
- Tailscale documentation: policy file syntax, grants; Kubernetes operator compatibility and overview; Headscale issue #1202 on operator compatibility; Charmhub tailscale-k8s page.
- Railway documentation (TCP proxy) and community threads confirming no inbound UDP.
- Items marked [verify] come from general knowledge or were not confirmed in these sources.